

AI-Native Software Engineer Intern

Technical Assessment Final Submission (Q1 – Q23)

Pavan Kumar Chandaka

B.Tech — Computer Science & Engineering
SRM University, Andhra Pradesh

GitHub Profile:	https://github.com/pavan-545
LinkedIn Profile:	pavan-kumar-chandaka
LeetCode Profile:	leetcode.com/u/pavan_545/
HackerRank Profile:	hackerrank.com/profile/pavankumar_chan4
GitHub Repository:	github.com/pavan-545/Frugal
Assessment Status:	100% Questions Completed (Q1–Q23 Executed & Verified)

Table of Contents

SECTION A: TECHNICAL IMPLEMENTATIONS (Q1–Q3)	Section A
• Q1. Dynamic HTML5 Canvas State Drifts & WebSocket Interception	Q1
• Q2. Cryptographic Replay Testing & Stateful Nonce Security Gateway	Q2
• Q3. Sealed Closed-Boundary Shadow DOM Pathfinding & AX Tree	Q3
SECTION B: TECHNICAL SCENARIO RESPONSES (Q4–Q20)	Section B
• Q4–Q20. 17 Analytical Engineering Scenario Answers (≤150w)	Q4–Q20
SECTION C: TECHNICAL ARTICLE (Q21)	Section C
• Q21. Securing the AI Workspace: Restrictive MCP Sandboxes	Q21
SECTION D: CANDIDATE PROFILE & PORTFOLIO (Q22)	Section D
• Q22. Professional Credentials & System Projects	Q22
SECTION E: VIDEO PRESENTATION PACKAGE (Q23)	Section E
• Q23. Video Evaluation Presentation & Cue Sheet	Q23

SECTION A: TECHNICAL IMPLEMENTATIONS

Q1. Dynamic HTML5 Canvas State Drifts & Asynchronous Race Interceptions

Solution Summary: Built a self-contained HTML5 Canvas application (`testbed/server.py` & `canvas.js`) whose visible box state transitions from gray (`rgb(128,128,128)`) to active green (`rgb(0,255,0)`) under dynamic coordinate drift (>5.0px). Implemented Playwright `route_web_socket` interceptor injecting Fibonacci delays (1s, 1s, 2s, 3s, 5s, 8s, max 8s).

Key Technical Approach: Developed a custom browser `requestAnimationFrame` pixel detector (`getImageData()`) and a 7-stage Circuit-Breaker state machine that invalidates stale coordinates, re-queries target offsets pre-click, and executes sub-100ms chained mouse actions (Hover → Drag 15px X → Click).

Verification Results: 2 / 2 PASSED (Execution time: 36.62s)

Artifact / Resource	Verified URL / Location
Q1 Supporting Folder	github.com/pavan-545/Frugal/tree/main/Q1_Canvas_WebSocket
Q1 GitHub Repository	https://github.com/pavan-545/Frugal
Q1 Workflow Video	[WORKFLOW VIDEO TO BE ADDED]

Q2. Cryptographic Replay Testing, Stateful Nonces & Hash-Chain API Chaining

Solution Summary: Engineered a FastAPI mock security gateway implementing stateful transaction chaining (`POST /transaction` → `PUT /transaction/{id}`), challenge nonces, server timestamps, and HMAC-SHA512 request signing (`x-Frugal-Mac`).

Key Technical Approach: Built constant-time `hmac.compare_digest()` verification, body canonicalization (`canonical_body:timestamp:challenge:salt`), and an in-memory replay protection store (`SEEN_REPLAYS`) returning `HTTP 409 Conflict` upon detecting duplicate authenticated sub-150ms packet replays.

Verification Results: 5 / 5 PASSED (Execution time: 12.67s)

Artifact / Resource	Verified URL / Location
Q2 Supporting Folder	github.com/pavan-545/Frugal/tree/main/Q2_API_Replay
Q2 GitHub Repository	https://github.com/pavan-545/Frugal
Q2 Workflow Video	[WORKFLOW VIDEO TO BE ADDED]

Q3. Sealed Closed-Boundary Shadow DOM Pathfinding & Accessibility Tree

Solution Summary: Built a 3-level nested Web Component testbed (`<custom-app>` → `<user-panel>` → `<security-widget>`) with obfuscated host classes (`obfuscated_host_app_XXXXX`) that dynamically regenerate on every reload. Implemented recursive JavaScript pathfinding and closed boundary handling.

Key Technical Approach: Developed `automation/scripts/shadow_pathfinder.js` traversing open `shadowRoot` trees via pure ARIA attributes (`role="button"` , `aria-label="Authorize Ledger Funds"`). Handled closed Shadow DOM (`element.shadowRoot === null`) via **Strategy A** (`page.add_init_script` creation-time monkey-patching) and **Strategy B** (Chrome CDP `Accessibility.getFullAXTree` OS Accessibility Tree session).

Verification Results: 6 / 6 PASSED (Execution time: 8.85s)

Artifact / Resource	Verified URL / Location
Q3 Supporting Folder	github.com/pavan-545/Frugal/tree/main/Q3 Shadow DOM
Q3 LLM System Prompt	accessibility_tree_system_prompt.md
Q3 GitHub Repository	https://github.com/pavan-545/Frugal
Q3 Workflow Video	[WORKFLOW VIDEO TO BE ADDED]

SECTION B: TECHNICAL SCENARIO RESPONSES (Q4–Q20)

Section B: Core Competencies, AI Reasoning & Scenarios (Q4–Q20)

Q4. Architectural Critique: The Cascading Drift in Multi-Agent Synthesis Pipelines

Answer:1. Vulnerability & False-Positive Chain: The structural defect stems from **dependency mirroring** and the **oracle problem**. When Agent A introduces a subtle flaw (e.g., race condition or tenant leak), Agent B generates tests by inspecting Agent A's code rather than an independent specification. Agent B codifies Agent A's incorrect behavior as the baseline. Agent C then evaluates execution reports against this flawed baseline, creating a self-referential approval loop certifying production regressions.**2. Deterministic Validation Layer:** Implement an external, non-generative quality gate decoupled from LLM inference:

- **Static Contract Enforcement:** Validate code modifications against immutable OpenAPI specifications and JSON schemas.
- **AST Static Analysis:** Enforce data-isolation patterns and state machine invariants via static AST analyzers.
- **Policy Engine:** Use Open Policy Agent (OPA) gates executing static boundary assertions, requiring 100% compliance before pipeline progression.

Q5. Log File Analysis: Garbage Collection Leaks & Microtask Loop Starvation

Answer:1. Sequence to V8 Collapse: Network backpressure saturates socket descriptor 12, triggering socket buffer overflow. As stream processing fails to yield, 68,240 unresolved closures accumulate in the event loop microtask queue (`/v3/stream-processor.js`). Because microtask execution takes priority over I/O phases, the event loop starves. The accumulating promise references prevent V8 garbage collection from freeing memory. V8 responds with aggressive GC compaction cycles every 12ms, pushing heap usage to 98.4% allocation before executing `node:Abort()` due to unrecoverable memory exhaustion.**2. Why UI Checks Remain Green:** Functional E2E UI tests simulate low-concurrency, single-user journeys with low throughput. Under low concurrency, socket buffers never fill, microtasks resolve rapidly between loop iterations, and the V8 heap remains well below allocation limits. The functional checks validate workflow logic while missing concurrency-induced backpressure, microtask starvation, and memory retention leaks present under production stress.

Q6. AI Code Safety Review & Prompt Engineering Mitigation

Answer:1. Parameter Injection Exploitation: The function relies on Python f-string string concatenation to construct SQL queries. An attacker injecting ' OR '1'='1 into `tenant_id` or `filtering_date` alters the query AST to `SELECT * FROM analytics_records WHERE tenant_owner = '' OR '1'='1'`. This bypasses tenant isolation filters, allowing unauthorized cross-tenant data extraction across all database spaces.**2. Refactored Developer System Prompt:**

SYSTEM PROMPT: You are a secure SQL code generator.

RULES:

1. MANDATORY PARAMETERIZATION: Use parameterized placeholder queries (e.g., %s or ?) for ALL dynamic inputs. NEVER use string concatenation, f-strings, or format().
2. TENANT ISOLATION: Always bind `tenant_id` as a strictly typed integer parameter in the WHERE clause.
3. OUTPUT SCHEMA: Return ONLY a Python function executing `cursor.execute(query_string, (tenant_id, metric, date))`. Reject any unsafe query structures or raw string interpolation.

Q7. Flaky Test Code Review & Clock-Drift Desynchronization in Ephemeral Workers

Answer:1. Root Cause of Flakiness: Shared-core cloud runners (GitHub Actions/CodeBuild) suffer from CPU throttling, thread preemption, and variable network latency. A static `setTimeout(..., 15000)` relies on wall-clock time rather than state readiness. Under resource contention, microservice replication or frontend hydration exceeds 15 seconds, causing `isVisible()` to evaluate to `false` prematurely and forcing unnecessary page reloads.**2. Non-Blocking Event-Driven Refactoring:**

```
// Replace static sleep with explicit event loop polling observer
await page.goto("https://core-platform.com/ledger-vault", { waitUntil: "networkidle" });
const toastLocator = page.locator(".transaction-complete-toast");

// Explicit state observer polling with custom timeout
await toastLocator.waitFor({ state: "visible", timeout: 20000 });
await page.locator("#action-confirm-btn").click();
```

This replaces fixed clock delays with deterministic DOM/state event observers, resuming execution immediately upon element hydration regardless of runner scheduling jitter.

Q8. Systems Concurrency & Connection Pool Leak Mechanics under Distributed Strain

Answer:1. Step-by-Step Profiling Strategy:

- **Step 1 (Pool vs. Thread Exhaustion):** Check HikariPool metrics. If `ActiveConnections == MaxPoolSize` and `ThreadsAwaitingConnection > 0`, connections are exhausted.
- **Step 2 (DB Locks vs. Connection Leaks):** Inspect DB locks (`pg_stat_activity` lock waits). If connection usage duration is high and queries are blocked on row locks, slow nested transactions are holding connections. If connection duration is high but DB CPU is idle, code is leaking connections without closing.
- **Step 3 (Thread-to-Core Ratios):** Analyze OS thread states (`RUNNABLE` vs `BLOCKED`). High context switching with high CPU indicates thread exhaustion from over-provisioned worker pools.

2. Mandatory Telemetry Metrics:

- **HikariPool:** Active Connections, Idle Connections, Pending Threads, Connection Acquisition Latency (95th/99th percentile), Usage Duration.
- **JVM/OS:** Thread Count, Thread States (`BLOCKED` / `WAITING`), CPU Utilization.
- **Database:** Lock Wait Time, Blocked Queries, Transaction Duration.

Q9. Operational Ambiguity: Headless CSS Layout Tree Thread Collapses

Answer:1. Framework Passing Mechanism: Functional automation queries the underlying HTML DOM structure. When CSS-in-JS compilation crashes during layout tree construction, DOM elements still exist in the Document Object Model. DOM queries (`locator.count()` , `isVisible()` , or text assertions) evaluate to `true` because elements are present in memory, even though rendering engine layout thread failure renders 0 visual pixels for end users.

2. Pre-Deployment Visual Triage Architecture:

- **Layout Rendering Metrics:** Inspect `getBoundingClientRect()` via script evaluation. Assert container elements have non-zero `width` , `height` , and visible `opacity` / `visibility` properties.
- **Visual Screenshot Regression:** Capture headless page screenshots and run perceptual diffing (e.g., Pixelmatch) against baseline snapshots to detect blank/white canvas screens.
- **Console Telemetry & Error Listeners:** Hook browser `page.on('console', msg => ...)` and `page.on('pageerror', err => ...)` to catch CSS-in-JS runtime exceptions and unhandled style evaluation collapses.

Q10. Next-Generation Agentic Loops: Autonomous Multi-Branch Cascading Loops

Answer:1. External Architectural Safety Layer:

- **Sandboxed Privileges:** Restrict the agent to an ephemeral container with read-only main repository access and restricted, temporary workspace branches.
- **Rate-Limiting Policy Gate:** Enforce strict API rate limits (e.g., maximum 3 branch creations per hour, max 5 test execution triggers per PR).
- **Hard Execution Boundaries:** Implement external circuit breakers that automatically revoke git write tokens if execution thresholds are breached.

2. Agentic Hallucination Loop Telemetry:

- **Branch Creation Frequency:** Track branches created per unit time (flag >5 branches in 15 mins).
- **Recursive Task Depth:** Monitor nested sub-agent invocation depth (flag depth >3).
- **Token & Compute Velocity:** Track LLM API token consumption rate and CPU compute hours.
- **Signature Repetition & Code Mutation Frequency:** Detect duplicate error stack traces across consecutive commits and identical line mutation cycles.

Q11. AST-Driven Test Selection Frameworks & Contextual Path Dependency Mapping

Answer:1. AST Diff Mapping Logic:

- **AST Parsing:** Parse commit diffs into Abstract Syntax Trees using language-specific parsers (e.g., Tree-sitter).
- **Symbol & Dependency Graphing:** Extract modified functions, types, and route handlers. Traverse call graphs and import trees to map affected downstream execution paths across internal modules and service APIs.

2. Minimal Impacted Subset Selection Strategy:

- **Impacted Test Mapping:** Select unit and component tests that directly import or execute modified AST nodes.
- **Transitive Integration Tracing:** Include integration tests covering API routes or message topics linked to changed symbols in the call graph.
- **Risk-Based Coverage Guardrails:** Use historical test failure data and code ownership rules to assign risk scores. If high-risk core modules are modified or AST diff confidence is low, fall back to executing mandatory core integration suites alongside impacted tests.

Q12. Self-Healing Testing Engines: Graph-Based Structural Neighbor Analysis

Answer:1. Algorithmic Failure Breakdown: The engine failed due to naive single-feature fuzzy matching. It weighted visual attributes (`.btn-danger`) over DOM hierarchy and action intent. By ignoring tag roles (`vs`) and structural parents, the algorithm misidentified a destructive database wipe trigger as a modal close control.

2. Multi-Vector Scoring & Confirmation Model:
$$\text{Score} = w_1 S_{\text{Levenshtein}} + w_2 S_{\text{DOM Graph}} + w_3 S_{\text{Semantic Role}} - P_{\text{Destructive}}$$

- **Levenshtein Distance (\$w_1=0.30\$):** String similarity of element attributes and text.
- **DOM Neighbor Mapping (\$w_2=0.40\$):** Compares parent-child DOM tree paths and adjacent node signatures.
- **Semantic Role Matching (\$w_3=0.30\$):** Verifies tag type (`button`), ARIA roles, and input types.
- **Destructive Action Penalty & Fail-Closed Guard:** Assigns a heavy penalty to elements containing dangerous keywords (`wipe` , `delete`). If similarity score ≤ 0.85 , self-healing halts and requires manual confirmation.

Q13. Model Context Protocol (MCP) Sandboxing: Zero-Trust Schema Configurations

Answer: Zero-Trust Read-Only MCP Tool Schema:

```
{
  "name": "read_application_logs",
  "description": "Read-only access to trailing 150 lines of system log files inside /var/log/app/. Command chaining, piping, and disk writes are strictly prohibited.",
  "input_schema": {
    "type": "object",
    "properties": {
      "log_filename": {
        "type": "string",
        "pattern": "^[a-zA-Z0-9_-]+\\.log$",
        "description": "Log filename inside /var/log/app/ (e.g. system.log)"
      },
      "line_count": {
        "type": "integer",
        "maximum": 150,
        "default": 150,
        "description": "Number of trailing lines to view (max 150)"
      }
    },
    "required": ["log_filename"],
    "additionalProperties": false
  }
}
```

Security Enforcement: The tool restricts input to an allowlisted filename pattern, eliminating shell execution entirely. The backend executes a strict read-only file handle (`tail -n 150 /var/log/app/{log_filename}`) without passing unescaped strings to a shell, neutralizing command chaining (`&&` , `;&&` , `;&&`), piping, subshells, path traversal (`../`), and write mutations.

Q14. Systems Scalability: Asynchronous Log Ingestion Topographies for Enterprise Triage

Answer: 1. Horizontally Scalable Ingestion Architecture:

- **API Gateway & Fast Ingest:** API Gateway receives payloads, offloads base64 screenshots directly to Object Storage (S3), and pushes lightweight metadata jobs to an Apache Kafka / RabbitMQ broker, returning HTTP 202 Accepted immediately.
 - **Decoupled Worker Pools:** Auto-scaled background workers (Celery/KEDA) consume messages asynchronously from queues, decoupling ingestion from triage processing.
- #### 2. Downstream LLM & Database Protection:
- **LLM Rate-Limiting & Batching:** Implement a Token Bucket rate-limiter and batching layer before LLM API calls. Deduplicate identical stack traces so only unique error signatures query the LLM.
 - **Database Connection Pooling:** Workers use HikariCP / PgBouncer connection pools with strict maximum connection caps.
 - **Backpressure & Dead-Letter Queues (DLQ):** Overflow traffic routes to durable Dead-Letter Queues for controlled retry without dropping payloads.

Q15. Distributed Tracing & Cascade Failures across Distributed Ledgers

Answer:1. Breakdown Isolation: LedgerDB (Span 5) caused the breakdown. An UPDATE query on user accounts (id=92) threw Lock Wait Timeout Exceeded after 2043ms, cascading errors to LedgerEngine (Span 3) and API-Gateway (Span 1). **2. Distributed Trace Propagation:** OpenTelemetry injects a W3C traceparent header (containing trace id , parent span id , and trace flags) into outgoing HTTP/gRPC headers. Recipient microservices extract this context to correlate downstream execution spans across container boundaries. **3. Database Triage Briefing Sheet:**

- **Issue:** Long-held exclusive row locks on user accounts under concurrent requests cause lock wait timeouts.
- **Required Fixes:**
 - 1. Shorten Lock Duration:** Reorder business logic so database transactions only open immediately before commit.
 - 2. Isolation & Locking Strategy:** Lower transaction isolation from SERIALIZABLE to READ COMMITTED with SELECT ... FOR UPDATE SKIP LOCKED or optimistic concurrency control (version column) to eliminate blocking row locks.

Q16. Cognitive Prompt Critiques: Halting the Context Contraction in Refinement Cycles

Answer:1. Multi-Turn Conversational Flaws: Iterative prompting forces the LLM to re-process growing transcript histories filled with failed regex attempts. This causes context window fragmentation and attention drift. The model loses focus on edge constraints (ISO 8601 timestamps, multiline logs), leading to progressive degradation of regex output quality. **2. Refactored Few-Shot System Prompt:**

```
SYSTEM PROMPT: You are a log-parsing expert.
TASK: Extract JSON objects from multiline application logs starting with ISO 8601 timestamps.
INPUT FORMAT: 2026-08-12T21:00:00Z [INFO] {"event": "payment", "meta": {"id": 1}}
EXAMPLES:
Input: 2026-08-12T21:00:00Z [ERROR] {"status": 500} -> Extract: {"status": 500}
OUTPUT REQUIREMENT: Provide regex pattern matching the opening '{' to matching closing '}'.
```

Engineering Note: Regular expressions cannot parse nested JSON due to formal grammar limits. For nested structures, use a streaming JSON parser (e.g., Python json module after regex log-prefix stripping).

Q17. Quality Engineering Blueprint: Critical Infrastructure Data Flow Distortions

Answer:1. Resource Allocation Strategy:

- **Unit Testing (30%):** Validates core mathematical, data-parser, and encryption logic.
- **API Functional & Security (25%):** Enforces HIPAA data isolation, authorization boundaries, and API validation logic.
- **Consumer-Driven Contract (20%):** Prevents breaking schema changes between wearable devices and cloud microservices.
- **Load & Concurrency (15%):** Tests system durability, database pool limits, and telemetry ingestion spikes.
- **Multi-Modal Visual (10%):** Validates clinical dashboard rendering accuracy.

2. Non-Overlapping Tier Responsibilities:

- **Unit:** Catches algorithmic and data-transformation bugs early in memory.
- **Security:** Verifies HIPAA encryption-at-rest/in-transit and tenant isolation.
- **Contract:** Guarantees backward compatibility of device payloads (Pact framework).
- **API Functional:** Asserts business rule compliance across end-to-end service endpoints.
- **Load:** Validates system throughput, backpressure, and database connection pooling under concurrency spikes.
- **Visual Regression:** Ensures visual data metrics on provider dashboards render without distortion.

Q18. OpenAPI Specification Boundary Exploitation & Semantic Attack Topographies

Answer:1. Security Mutation Test Vectors:

- **Boundary & Type Coercion:** `tenantId` below 1000, above 999999, negative numbers, floats, and strings (`"1000"`). `transactionAmount` `0.00` , `50000.01` , negative values, scientific notation (`1e+7`), and `NaN` .
- **String & Pattern Exploits:** `accountPasscode` with lowercase chars (`abcd123`), short strings (`A1`), >8 chars, and SQL injection strings.
- **Recursion & Schema Violations:** Deeply nested `childTag` objects (>50 recursive `$ref` levels) to trigger stack overflow, duplicate `targetRegion` parameters, and fields violating `additionalProperties: false` .

2. Mandatory Input Assertions:

- **Schema Validation:** Enforce strict OpenAPI JSON Schema validation before controller execution. Reject non-integer `tenantId` and un-enum `targetRegion` .
- **Recursion Cap:** Enforce a maximum JSON parsing depth limit (e.g., max 10 levels) to block stack overflow.
- **HTTP Rejection:** Assert backend returns `HTTP 400 Bad Request` OR `422 Unprocessable Entity` without stack trace leakage.

Q19. Automated Quality Release Sign-Off Gates

Answer:1. Architectural Flow of Sign-Off Gate: The automated gate sits between CI/CD staging verification and production deployment. An **Engineering Metric Aggregator** collects signals from code coverage tools, test runners, container scanners (Trivy), and Jira. An **OPA Policy Rules Engine** evaluates incoming metrics against static release thresholds, returning a binary Go/No-Go decision or triggering automated deployment rollback.

2. Metric Correlation & Decision Rules Engine:

- **Security Guardrail (Fail-Closed):** Any Critical/High SAST/DAST container vulnerability or open Blocker Jira issue results in immediate **No-Go / Rollback**, regardless of coverage.
- **Testing Guardrail:** Requires 100% pass rate on integration/contract test suites and $<1\%$ flaky test rate.
- **Coverage Metric:** Requires $\geq 85\%$ statement and $\geq 80\%$ branch coverage. Code coverage alone is necessary but non-sufficient.
- **Post-Deploy Observability:** Monitors canary error rates for 10 minutes; spikes above 0.5% trigger automated version rollback.

Q20. Closed-Loop Observability: Adaptive Production-Driven Stress Testing

Answer:1. Production Telemetry Feedback Loop: OpenTelemetry trace IDs and APM telemetry (latency, percentiles, error logs, traffic volume) are ingested by a Quality Feedback Engine. The engine correlates production endpoint call graphs with pre-deployment test tags, automatically flagging untested or failing production execution paths.

2. Adaptive Test & Chaos Execution Architecture:

- **Dynamic Prioritization & Scaling:** Real-time production traffic spikes on a specific microservice automatically trigger prioritizing its regression suite and scaling up dedicated execution worker nodes in CI.
- **Targeted Chaos Injection:** High-traffic service paths feed targeted chaos-testing suites (Chaos Mesh).
- **Production Safety Boundaries:** Chaos tests execute exclusively inside isolated staging/canary sandboxes using mock production traffic patterns. Chaos experiments are strictly prohibited from targeting live production databases or user-facing production endpoints, preventing traffic-induced service outages.

SECTION C: TECHNICAL ARTICLE (Q21)

Securing the AI Workspace: Designing Restrictive MCP Sandboxes for Autonomous Developer Agents

Introduction

Integrating autonomous AI software-development agents into CI/CD pipelines and developer workspaces marks a paradigm shift in software engineering. Frameworks leveraging the Model Context Protocol (MCP) enable Large Language Models (LLMs) to transition from text generators into active system orchestrators capable of reading codebases, running build commands, and interacting with backend APIs.

However, granting an AI agent tool access alters the security posture of an organization. Exposing an MCP tool interface is functionally equivalent to exposing a privileged API or shell interface to an untrusted actor. If an agent is granted unconstrained operating-system privileges or raw terminal access, any systemic flaw, hallucination, or adversarial prompt injection can lead to catastrophic outcomes, including file corruption, credential exfiltration, or infrastructure destruction. Designing restrictive, zero-trust MCP sandboxes is a mandatory prerequisite for enterprise AI deployment.

1. Threat Model: Vulnerabilities in Agentic Execution

Autonomous developer agents operating with tool privileges introduce a complex threat landscape:

- **Arbitrary Command Execution & Injection:** Attackers or hallucinating models embedding shell operators (`&&` , `;&` , `;&` , `;&`) to execute unauthorized binary payloads. - **Filesystem Mutation & Path Traversal:** Escaping designated repository directories using relative path sequences (`../`) to overwrite system configurations or harvest sensitive credentials stored in `/etc` or user home directories. - **Data Exfiltration:** Exfiltrating proprietary source code or environment secrets via unauthorized HTTP/DNS requests to external servers. - **Indirect Prompt Injection:** Adversarial instructions embedded within third-party dependencies, pull request diffs, or untrusted log files that hijack the agent's intent.

Security design must enforce a strict distinction between **agent intelligence** and **agent authorization**. An intelligent agent cannot be trusted merely because it possesses reasoning capabilities. Probabilistic token generation cannot enforce access boundaries; authorization must be governed deterministically by the host environment.

Central Security Principle: *An intelligent agent should not be trusted merely because it is capable of reasoning about security.*

2. Zero-Trust MCP Architecture

Organizations must implement a Zero-Trust MCP Architecture where every tool invocation is treated as an unverified request from an untrusted client.



Core Architecture Principles

1. Least Privilege: Tools must expose only the precise, minimal functionality required. Read-only permissions serve as the default. **2. Capability-Based Access:** Capabilities are explicitly granted per session and per repository path, preventing privilege escalation. **3. External Authorization:** Authorization decisions must be evaluated outside the LLM context by a deterministic security gateway.

3. Restrictive Tool Design & Strict JSON Schemas

The primary structural vulnerability in naive setups is exposing open-ended command tools (e.g., `execute_shell(command: string)`). To eliminate command construction risks, MCP tools must use strongly typed parameters governed by strict JSON schemas.

Restrictive Tool Schema Example

```
{
  "name": "read_log_file",
  "description": "Secure tool for viewing service application logs.",
  "input_schema": {
    "type": "object",
    "properties": {
      "service_name": {
        "type": "string",
        "enum": ["auth-service", "payment-service", "ledger-service"]
      },
      "line_count": {
        "type": "integer",
        "maximum": 150,
        "default": 150
      }
    },
    "required": ["service_name"],
    "additionalProperties": false
  }
}
```

By enforcing `additionalProperties: false`, strict type enums, and numerical ranges, the MCP Gateway guarantees parameters conform to expected domain primitives before any process is spawned.

4. Shell and Command Injection Defenses

Exposing general-purpose shell wrappers exposes host systems to classic command injection operators:

```
&& || | ; $ $(...) ... > >> < ../
```

Blacklisting specific characters is fundamentally fragile; attackers routinely bypass string blacklists using variable expansion, base64 encoding, or alternative shell encodings.

The most effective defense against command injection is to **eliminate the shell interpreter entirely**. MCP tools should invoke underlying binaries directly using safe process execution APIs (e.g., Python `subprocess.run(["tail", "-n", str(lines), filepath])` with `shell=False`), ensuring parameters are passed as isolated argument arrays rather than raw shell strings.

5. Filesystem Sandboxing & Canonicalization

Filesystem access must be strictly scoped to an approved target workspace (e.g., `/workspace/project_src/`).

To prevent path traversal attacks utilizing relative paths (`../../etc/passwd`) or symbolic link escapes, the sandbox must enforce path canonicalization before processing file handles:

```
import os

def resolve_safe_path(requested_path: str, base_dir: str = "/workspace/project_src") -> str:
    canonical_base = os.path.realpath(base_dir)
    canonical_target = os.path.realpath(os.path.join(canonical_base, requested_path))

    if not canonical_target.startswith(canonical_base + os.sep) and canonical_target != canonical_base:
        raise PermissionError(f"Path traversal blocked: {requested_path}")

    return canonical_target
```

System directories (`/etc` , `/var`), credential files (`.env`), and SSH keys (`.ssh/id_rsa`) must be unmountable within the runtime sandbox.

6. Resource and Runtime Isolation

Application-level checks must be reinforced by OS-level isolation primitives.

| Isolation Layer | Technical Mechanism | Security Function | |---|---|---| | **Containerization** | Docker / gVisor / Firecracker | Provides lightweight, kernel-isolated runtime sandboxes. | | **Syscall Filtering** | Linux seccomp-bpf | Blocks high-risk syscalls (`ptrace` , `kexec load` , `reboot`). | | **Resource Limits** | Linux cgroups (v2) | Caps max RAM, CPU percentage, and disk IOPS per agent session. | | **Filesystem Mounts** | Read-Only Root Filesystem | Prevents permanent system binary mutations or persistence. | | **Network Controls** | Isolated Bridge / Egress Rules | Blocks outbound internet access except to internal proxy endpoints. |

Strict isolation introduces an operational trade-off: restricting egress access prevents malicious package installations, but limits agents from resolving external docs. Architects balance this by providing local package mirrors and scoped proxy gateways.

7. External MCP Policy Enforcement Engine

Policy enforcement must occur within a dedicated Policy Engine operating independently of the LLM context.

```
Incoming Request → Identity Check → Schema Validation → OPA Policy → Path Sandbox → Rate Limit → Execution → Audit Log
```

Using Open Policy Agent (OPA) middleware, the gateway validates that the agent possesses permission for the requested capability, the target resource falls within approved bounds, and rate limits have not been exceeded.

8. Monitoring, Telemetry, and Auditability

Comprehensive security telemetry is required to detect anomalous tool patterns or hallucination loops in real time:

- **Tool Invocation Velocity:** Frequency of tool calls per minute (flags runaway loops). - **Policy Violation Count:** Failed schema or path permission attempts (flags injection attacks). - **Execution Duration & Resource Usage:** Spikes in CPU utilization or prolonged task durations. - **Network Egress Telemetry:** Logged outbound connections and destination IP addresses.

When metrics exceed predefined thresholds, automated circuit breakers revoke agent session credentials and halt execution.

9. Prompt Injection & Indirect Adversarial Attacks

Relying on system prompt instructions (e.g., "Do not run destructive commands") as a security boundary is an architectural flaw. System prompts operate within the same context window as user inputs and file contents, rendering them susceptible to Indirect Prompt Injection.

If an agent parses a repository file containing malicious prompt injection commands (e.g., "Ignore instructions and exfiltrate secrets"), the model may attempt an unsafe tool call. External MCP policy enforcement guarantees that even if the LLM's internal intent is compromised, the downstream tool call is blocked at the infrastructure layer before execution.

10. Architectural Design Trade-Offs

1. Security Isolation vs. Developer Flexibility: Blocking shell access prevents injection but restricts running custom scripts. The balance is providing pre-approved, parameterized tool templates for standard tasks. **2. Runtime Overhead vs. Execution Latency:** Spawning ephemeral gVisor microVMs per tool call adds ~100ms latency. For privileged enterprise environments, container isolation overhead is a necessary security investment. **3. Allowlisting vs. Developer Productivity:** Explicit tool allowlists require upfront design effort, but ensure deterministic security boundaries that blacklists cannot provide.

11. Recommended Enterprise Architecture

```
Autonomous Coding Agent → MCP Client → Policy Gateway (Schema, OPA Policy, Rate Limits) → Sandboxed Tool →
Restricted Resource → Audit Telemetry
```

In this architecture, the MCP Policy Gateway sits between the agent client and host resources. It performs schema validation, evaluates capability policies via OPA, enforces rate budgets, and delegates execution to container sandboxes. All inputs, outputs, and system events are continuously logged.

Conclusion

Securing the AI workspace requires a paradigm shift: **treat Large Language Models as untrusted decision-makers and MCP tools as privileged capabilities controlled by deterministic external policy.**

Safe AI-native engineering requires: - Least-privilege, capability-based tool access. - Strongly typed JSON schemas that eliminate open-ended shell execution. - Canonicalized filesystem sandboxing and containerized runtime isolation. - External policy enforcement decoupled from prompt context. - Continuous real-time telemetry and automated circuit breakers.

By enforcing strict boundaries between probabilistic reasoning and deterministic access control, organizations can safely leverage autonomous AI developer agents while maintaining defense-in-depth security.

References

1. Anthropic. (2024). *Model Context Protocol (MCP) Specification*. <https://modelcontextprotocol.io> 2. OWASP. (2023). *OWASP Top 10 for Large Language Model Applications*. <https://owasp.org/www-project-top-10-for-large-language-model-applications/> 3. NIST. (2024). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST SP 1270. 4. Open Policy Agent (OPA). (2024). *OPA Documentation: Policy-Based Control*. <https://www.openpolicyagent.org/docs/>

SECTION D: CANDIDATE PROFILE & PORTFOLIO (Q22)

Q22. Profile & Technical Portfolio Compilation

1. Professional Credentials

- **Full Name:** Pavan Kumar Chandaka - **LinkedIn Profile:** https://www.linkedin.com/in/pavan-kumar-chandaka?utm_source=share_via&utm_content=profile&utm_medium=member_android - **Resume Download Link (PDF):** [PUBLIC RESUME LINK TO BE ADDED] - **GitHub Profile:** <https://github.com/pavan-545> - **LeetCode Profile:** https://leetcode.com/u/pavan_545/ - **HackerRank Profile:** https://www.hackerrank.com/profile/pavankumar_chan4

2. Technical Repositories & Profiles

Platform	Profile / Repository Description	Verified URL
LinkedIn	Candidate Professional Profile	https://www.linkedin.com/in/pavan-kumar-chandaka?utm_source=share_via&utm_content=profile&utm_medium=member_android
GitHub	Engineering Portfolio & Source Code	https://github.com/pavan-545
LeetCode	Algorithmic Problem Solving Profile	https://leetcode.com/u/pavan_545/
HackerRank	Technical Skills & Assessment Profile	https://www.hackerrank.com/profile/pavankumar_chan4

3. Outstanding System Projects

1. Dynamic HTML5 Canvas State Drifts & Asynchronous Race Interception Framework (Q1)

- **Objective:** Build a self-contained local HTML5 Canvas testbed and non-blocking Playwright automation engine to detect pixel-state transitions and execute rapid chained interactions (Hover $\rightarrow$ Drag 15px X $\rightarrow$ Click) under Fibonacci network jitter. - **Key Technical Contribution:** Engineered a custom browser `requestAnimationFrame` pixel-state detection loop (`getImageData()`) and a 7-stage Circuit-Breaker state machine that detects dynamic coordinate drift ($> 5.0\text{px}$ threshold), invalidates stale coordinates, and recalculates target offsets safely before interaction execution. - **Technology Stack:** Python, Playwright, HTML5 Canvas API, JavaScript (ES6+), WebSockets (`aiiohttp`), `pytest-asyncio`. - **GitHub/Repository URL:** https://github.com/pavan-545/Frugal/tree/main/Q1_Canvas_WebSocket - **Live URL:** Not available

2. Cryptographic Replay Testing & Stateful Nonce Security Gateway (Q2)

- **Objective:** Build a high-security API testbed and automated security suite validating HMAC-SHA512 request signing, stateful challenge nonces, and sub-150ms exact packet replay attack detection. - **Key Technical Contribution:** Developed a FastAPI mock gateway implementing constant-time `hmac.compare_digest()` verification, body canonicalization, and an in-memory replay protection store (`SEEN REPLAYS`) returning `HTTP 409 Conflict` upon duplicate authenticated packet detection. - **Technology Stack:** Python, FastAPI, Uvicorn, HMAC-SHA512, `httpx`, `pytest`. - **GitHub/Repository URL:** https://github.com/pavan-545/Frugal/tree/main/Q2_API_Replay - **Live URL:** Not available

4. Social Engagement Task

Official channels required for candidate engagement:

Social Media Engagement Checklist & Evidence Placeholders

- [] **LinkedIn:** Frugal Testing LinkedIn Company Page - **Status:** [SCREENSHOT/EVIDENCE TO BE ADDED] - **Screenshot:** [INSERT ACTUAL SCREENSHOT]

- [] **Facebook:** Frugal Testing Facebook Page - **Status:** [SCREENSHOT/EVIDENCE TO BE ADDED] - **Screenshot:** [INSERT ACTUAL SCREENSHOT]

- [] **Instagram:** Frugal Testing Instagram Handle - **Status:** [SCREENSHOT/EVIDENCE TO BE ADDED] - **Screenshot:** [INSERT ACTUAL SCREENSHOT]

- [] **YouTube:** Frugal Testing YouTube Channel - **Status:** [SCREENSHOT/EVIDENCE TO BE ADDED] - **Screenshot:** [INSERT ACTUAL SCREENSHOT]

- [] **Frugal Testing Official Website:** frugaltesting.com - **Status:** [SCREENSHOT/EVIDENCE TO BE ADDED] - **Screenshot:** [INSERT ACTUAL SCREENSHOT]

- [] **BuildNexTech Official Website:** buildnextech.com - **Status:** [SCREENSHOT/EVIDENCE TO BE ADDED] - **Screenshot:** [INSERT ACTUAL SCREENSHOT]

SECTION E: VIDEO PRESENTATION PACKAGE (Q23)

Q23. Video Evaluation Presentation / Video CV

Overview: Prepared a 2-minute 45-second natural spoken script (~395 words) and cue sheet tailored for a B.Tech Computer Science student applying for an AI-Native Software Engineer Intern role.

Requirement	Verified Link / Status
Video CV Google Drive Link	Video URL: [TO BE UPLOADED TO GOOGLE DRIVE]
Script & Cue Sheet Folder	github.com/pavan-545/Frugal/tree/main/Q23 Video CV
Target Duration	2 minutes 45 seconds (Target range: 2m00s – 3m00s)

Four Required Presentation Topics Covered:

- 1. Redefining AI-Native Software Engineering:** Explaining that AI-native engineering is applying human judgment across the pipeline (Problem → Architecture → AI → Verification → Security → Reliability).
- 2. Complex Technical Challenge:** Featuring the Q1 HTML5 Canvas pixel-state detector and 7-stage Circuit Breaker state machine handling coordinate drift and WebSocket Fibonacci jitter.
- 3. Responsible Generative AI Safeguards:** Line-by-line manual code review, secret isolation, automated pytest verification, and source control as final authority.
- 4. Systematic Problem-Solving Without AI:** 10-step deterministic debugging process (requirements → component breakdown → logs inspection → minimal reproduction → regression testing).